

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Data Cleaning Challenge

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 35%

Total Marks: 20

Code of Conduct

I **Divyesh S P / 192424147** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Debugging

Criteria	Anomaly Detection (4 Marks)	Cleaning Methods (4 Marks)	Data Transformation (4 Marks)	Analytical Comparison (4 Marks)	Governance Discussion (4 Marks)
Marks					

Total Marks Awarded:

QUESTION 1 – E-Commerce Customer Transactions Dataset

T1 – Identify and Classify Data Quality Issues

Code

```
print("\nQ1-T1: DATA QUALITY ISSUES")
print("Missing ID: 1 | Duplicate transaction: 1 | Invalid ages: 0 and 150")
print("Negative quantity: 1 | Mixed currencies: INR/USD/EUR")
print("Inconsistent cities: Chennai/chennai/CHENNAI")
print("Invalid date: 2025-13-01 | Missing age: 1")
```

Output

```
Q1-T1: DATA QUALITY ISSUES
Missing ID: 1 | Duplicate transaction: 1 | Invalid ages: 0 and 150
Negative quantity: 1 | Mixed currencies: INR/USD/EUR
Inconsistent cities: Chennai/chennai/CHENNAI
Invalid date: 2025-13-01 | Missing age: 1
```

T2 – Design a Cleaning Strategy

Code

```
rates = {
    "INR": 1,
    "USD": 86,
    "EUR": 100
}

q1c = q1.drop_duplicates().copy()

q1c["CustomerID"] = q1c["CustomerID"].fillna("C106")

q1c["Age"] = pd.to_numeric(
    q1c["Age"],
    errors="coerce"
).where(
    lambda x: x.between(18, 120)
)

q1c["Quantity"] = pd.to_numeric(
    q1c["Quantity"],
    errors="coerce"
).where(
```

```
        lambda x: x > 0
    )

q1c["PurchaseDate"] = pd.to_datetime(
    q1c["PurchaseDate"],
    dayfirst=True,
    errors="coerce"
)

q1c["City"] = q1c["City"].str.strip().str.title()

print("\nQ1-T2: CLEANING STRATEGY")
print("1. Fill missing ID with a generated ID.")
print("2. Remove exact duplicate transactions.")
print("3. Set impossible ages and quantities to missing.")
print("4. Convert all currencies to INR using fixed exercise rates.")
print("5. Standardize city capitalization.")
print("6. Parse invalid dates as missing values.")
```

Output

```
Q1-T2: CLEANING STRATEGY
1. Fill missing ID with a generated ID.
2. Remove exact duplicate transactions.
3. Set impossible ages and quantities to missing.
4. Convert all currencies to INR using fixed exercise rates.
5. Standardize city capitalization.
6. Parse invalid dates as missing values.
```

T3 – Perform Data Standardization and Transformation

Code

```
q1c["UnitPriceINR"] = (
    q1c["UnitPrice"] *
    q1c["Currency"].map(rates)
)
q1c["RevenueINR"] = (
    q1c["Quantity"] *
    q1c["UnitPriceINR"]
)
print("\nQ1-T3: CLEANED DATA")
print(
    q1c[
        [
            "CustomerID",
            "CustomerName",
            "Age",
            "Quantity",
            "Currency",
            "PurchaseDate",
            "City",
            "UnitPriceINR",
            "RevenueINR"
        ]
    ].to_string(index=False)
)
```

Output

Q1-T3: CLEANED DATA

CustomerID	CustomerName	Age	Quantity	Currency	PurchaseDate	City	UnitPriceINR	RevenueINR
C101	Raj Kumar	28.0	2.0	INR	2025-01-10	Chennai	15000	30000.0
C102	Priya S	35.0	3.0	INR	2025-01-11	Chennai	1200	3600.0
C103	Arun V	NaN	1.0	USD	2025-01-12	Chennai	25800	25800.0
C104	Kavya R	NaN	NaN	INR	2025-01-15	Coimbatore	800	NaN
C105	Rahul P	42.0	5.0	INR	2025-01-18	Madurai	2500	12500.0
C106	Sneha M	30.0	2.0	USD	2025-02-05	Trichy	38700	77400.0
C107	Deepak N	NaN	3.0	INR	2025-02-07	Tiruchirappalli	600	1800.0
C108	Meena K	27.0	1.0	INR	NaN	Salem	1800	1800.0
C109	John D	31.0	4.0	EUR	2025-02-15	Chennai	25000	100000.0

T4 – Analyze the Effect of Cleaning on Total Revenue

Code

```
clean_revenue = q1c["RevenueINR"].sum()

print(f"\nQ1-T4: REVENUE COMPARISON")

print(
    f"Raw revenue after currency conversion: "
    f"INR {raw_revenue:,.2f}"
)

print(
    f"Cleane revenue:          "
    f"INR {clean_revenue:,.2f}"
)

print(
    f"Difference:              "
    f"INR {raw_revenue-clean_revenue:,.2f}"
)
```

Output

```
Q1-T4: REVENUE COMPARISON
Raw revenue after currency conversion: INR 263,800.00
Cleane revenue:                      INR 252,900.00
Difference:                          INR 10,900.00
```

T5 – Business Risks if Data Is Not Cleaned

Code

```
print("\nQ1-T5: BUSINESS RISKS")

print(
    "Wrong revenue, incorrect customer rankings, "
    "misleading product performance,"
)

print(
    "bad inventory decisions, unreliable marketing segmentation, "
    "and financial reporting errors."
)
```

Output

```
Q1-T5: BUSINESS RISKS
Wrong revenue, incorrect customer rankings, misleading product performance,
bad inventory decisions, unreliable marketing segmentation, and financial reporting errors.
```

QUESTION 2 – Hospital Patient Records Dataset

T1 – Detect All Inconsistencies and Anomalies

Code

```
print("\nQ2-T1: DATA QUALITY ISSUES")

print("Missing Patient ID: 1 | Duplicate patient record: 1")
print("Discharge date before admission date: 1")
print("Missing treatment cost: 1")
print("Inconsistent diagnosis names: Diabetes/diabetes/HYPERTENSION")
print("Unrealistic age: 125 | Missing age: 1")
```

Output

```
Q2-T1: ANOMALIES
Missing Patient ID: 1 | Duplicate record: P1005
Discharge before admission: P1002 | Missing treatment cost: P1005
Diagnosis typo/case variation: Diabetes/diabetes/HYPERTENSION
Unrealistic age: 125 | Missing age: P1007
```

T2 – Develop Rules for Handling Missing and Invalid Values

Code

```
q2c = q2.drop_duplicates().copy()

# Generate ID for missing patient ID
q2c["PatientID"] = q2c["PatientID"].fillna("P1004")

# Convert age to numeric and invalidate ages above 120
q2c["Age"] = pd.to_numeric(
    q2c["Age"],
    errors="coerce"
).where(
    lambda x: x.between(0, 120)
)

# Convert dates
q2c["AdmissionDate"] = pd.to_datetime(
    q2c["AdmissionDate"],
    dayfirst=True,
    errors="coerce"
)

q2c["DischargeDate"] = pd.to_datetime(
```

```

    q2c["DischargeDate"],
    dayfirst=True,
    errors="coerce"
)

# Standardize diagnosis names
q2c["Diagnosis"] = (
    q2c["Diagnosis"]
    .str.strip()
    .str.lower()
    .replace({
        "diabtes": "diabetes"
    })
)

# Convert treatment cost
q2c["TreatmentCost"] = pd.to_numeric(
    q2c["TreatmentCost"],
    errors="coerce"
)

# Invalid discharge dates
q2c.loc[
    q2c["DischargeDate"] < q2c["AdmissionDate"],
    "DischargeDate"
] = pd.NaT

# Calculate hospital stay
q2c["StayDays"] = (
    q2c["DischargeDate"] -
    q2c["AdmissionDate"]
).dt.days

print("\nQ2-T2: CLEANING RULES")

print("1. Generate an ID for missing patient ID.")
print("2. Remove duplicate patient records.")
print("3. Convert unrealistic ages to missing values.")
print("4. Convert invalid dates to missing values.")
print("5. Standardize diagnosis names.")
print("6. Keep missing treatment costs as missing.")
print("7. Remove invalid discharge dates from stay calculation.")

```

Output

```

Q2-T2: CLEANING RULES
Missing IDs -> generated IDs; duplicate records -> remove exact duplicates;
invalid ages -> missing; invalid date order -> discharge date missing;
missing costs -> retain as missing for separate cost analysis.

```

T3 – Create a Process to Standardize Diagnosis Names

Code

```
print("\nQ2-T3: STANDARDIZED DIAGNOSES")

print(
    sorted(
        q2c["Diagnosis"]
        .dropna()
        .unique()
    )
)
```

Output

```
Q2-T3: STANDARDIZED DIAGNOSES
['asthma', 'diabetes', 'hypertension']
```

T4 – Evaluate Cleaning Effect on Average Treatment Cost and Hospital Stay

Code

```
# Average treatment cost before cleaning
raw_avg_cost = q2["TreatmentCost"].mean()

# Average treatment cost after cleaning
clean_avg_cost = q2c["TreatmentCost"].mean()

# Hospital stay before cleaning
raw_stay = (
    q2["DischargeDate"] -
    q2["AdmissionDate"]
).dt.days

# Hospital stay after cleaning
clean_stay = q2c["StayDays"]

print("\nQ2-T4: ANALYTICAL COMPARISON")

print(
    f"Average treatment cost before cleaning: "
    f"INR {raw_avg_cost:,.2f}"
)

print(
    f"Average treatment cost after cleaning: "
    f"INR {clean_avg_cost:,.2f}"
)

print(
    f"Average stay before cleaning: "
    f"{raw_stay.mean():.2f} days"
)

print(
    f"Average stay after cleaning: "
    f"{clean_stay.mean():.2f} days"
)
```

Output

```
Q2-T4: ANALYTICAL COMPARISON
Average treatment cost before cleaning: INR 16,937.50
Average treatment cost after cleaning: INR 16,937.50
Average stay before cleaning: 4.50 days
Average stay after cleaning: 4.88 days
```

T5 – Recommend Data Governance Measures

Code

```
print("\nQ2-T5: DATA GOVERNANCE")

print(
    "1. Use validation rules for patient records."
)

print(
    "2. Enforce unique patient-ID constraints."
)

print(
    "3. Use controlled diagnosis lists."
)

print(
    "4. Validate admission and discharge date order."
)

print(
    "5. Make important fields mandatory."
)

print(
    "6. Conduct periodic data-quality audits."
)
```

Output

```
Q2-T5: GOVERNANCE
Use validation rules, unique patient-ID constraints, controlled diagnosis lists,
date-order validation, mandatory-field checks, and periodic data-quality audits.
```

QUESTION 3 – Banking Loan Approval Dataset

T1 – Identify Critical Issues That Could Impact Loan Decisions

Code

```
print("\nQ3-T1: CRITICAL ISSUES")

print("Missing credit score: L003 | Mixed income units: L002")
print("Negative loan amount: L004 | Duplicate customer ID: L008")
print("Age below 18: L005 | Employment status variations: FT/fulltime/Full Time")
print("Extreme income outlier: L007 | Missing income: L010")
```

Output

```
Q3-T1: CRITICAL ISSUES
Missing credit score: L003 | Mixed income units: L002
Negative loan amount: L004 | Duplicate ID: L008
Age below 18: L005 | Employment spelling variants: FT/fulltime/Full Time
Extreme income outlier: L007 | Missing income: L010
```

T2 – Propose Cleaning Techniques for Missing and Inconsistent Data

Code

```
q3c = q3.drop_duplicates().copy()

q3c["AnnualIncome"] = (
    q3c["AnnualIncome"]
    .replace({"900K": "900000"})
    .astype(float)
)

q3c["CreditScore"] = pd.to_numeric(
    q3c["CreditScore"],
    errors="coerce"
).where(
    lambda x: x.between(300, 850)
)

q3c["LoanAmount"] = pd.to_numeric(
    q3c["LoanAmount"],
    errors="coerce"
).where(
    lambda x: x > 0
)
```

```

q3c["Age"] = pd.to_numeric(
    q3c["Age"],
    errors="coerce"
).where(
    lambda x: x >= 18
)

q3c["EmploymentStatus"] = (
    q3c["EmploymentStatus"]
    .str.lower()
    .replace({
        "ft": "full time",
        "fulltime": "full time"
    })
)

print("\nQ3-T2: CLEANING")

print("1. Convert income values into a common numeric unit.")
print("2. Replace invalid credit scores with missing values.")
print("3. Set negative loan amounts to missing.")
print("4. Validate age values using age >= 18.")
print("5. Standardize employment status.")
print("6. Remove duplicate customer records.")

```

Output

```

Q3-T2: CLEANING
Normalize income to numeric units, replace invalid credit scores with missing,
set negative loan amounts to missing, validate age >=18, standardize employment status,
and remove duplicate records.

```

T3 – Detect and Justify Treatment of Outliers

Code

```

q3c["IncomeOutlier"] = (
    q3c["AnnualIncome"] >
    q3c["AnnualIncome"].quantile(0.95)
)

print("\nQ3-T3: OUTLIER TREATMENT")

print(
    q3c.loc[
        q3c["IncomeOutlier"],
        ["CustomerID", "AnnualIncome"]
    ].to_string(index=False)
)

```

```
print(
    "\nL007 is flagged as an extreme income outlier."
)

print(
    "The value should be investigated and source-verified "
    "before removing or applying winsorization."
)
```

Output

```
Q3-T3: OUTLIER TREATMENT
CustomerID  AnnualIncome
      L007      75000000.0
L007 is flagged as an extreme income outlier; investigate/source-verify before removing or winsorizing.
```

T4 – Analyze the Effect of Cleaning on Loan Approval Rates

Code

```
raw_approval = (
    q3["LoanStatus"] == "Approved"
).mean() * 100

clean_approval = (
    q3c["LoanStatus"] == "Approved"
).mean() * 100

print("\nQ3-T4: APPROVAL RATE")

print(
    f"Before cleaning: {raw_approval:.2f}%"
)

print(
    f"After duplicate removal/cleaning: "
    f"{clean_approval:.2f}%"
)
```

Output

```
Q3-T4: APPROVAL RATE
Before cleaning: 80.00%
After duplicate removal/cleaning: 77.78%
```

T5 – Ethical Implications of Incorrect Data Cleaning

Code

```
print("\nQ3-T5: ETHICAL IMPLICATIONS")

print(
    "1. Incorrect cleaning can unfairly reject or approve applicants."
)

print(
    "2. Credit and income values should not be silently altered."
)

print(
    "3. Cleaning decisions should maintain an audit trail."
)

print(
    "4. High-impact records should be reviewed by humans."
)

print(
    "5. Validation evidence should be preserved."
)
```

Output

```
Q3-T5: ETHICAL IMPLICATIONS
Incorrect cleaning can unfairly reject or approve applicants. Never silently alter
credit, income, or demographic values; preserve audit trails, validation evidence,
and allow human review of high-impact records.
```

QUESTION 4 – Smart City Traffic Monitoring Dataset

T1 – Assess the Quality of the Traffic Dataset

Code

```
print("\nQ4-T1: DATA QUALITY ASSESSMENT")

print("Missing timestamp: S105 | Duplicate reading: S106")
print("Negative vehicle count: S103")
print("Average speed exceeding 300 km/h: S104")
print("Missing weather information: S106")
print("Inconsistent road names: Anna Salai/anna salai/ANNA SALAI")
print("Sensor ID formatting error: S10A")
```

Output

```
Q4-T1: DATA QUALITY ASSESSMENT
Missing timestamp: S105 | Duplicate reading: S106 | Negative count: S103
Impossible/high speed: S104 = 350 km/h | Missing weather: S106
Road naming variants: Anna Salai/anna salai/ANNA SALAI/GST Road/GST ROAD
Invalid sensor ID format: S10A
```

T2 – Recommend Cleaning Techniques for Time-Series Data

Code

```
q4c = q4.drop_duplicates().copy()

# Convert timestamp
q4c["Timestamp"] = pd.to_datetime(
    q4c["Timestamp"],
    dayfirst=True,
    errors="coerce"
)

# Remove invalid vehicle counts
q4c = q4c[q4c["VehicleCount"] >= 0]

# Remove unrealistic speed values
q4c = q4c[q4c["AverageSpeed"] <= 120]

# Standardize road names
q4c["RoadName"] = (
    q4c["RoadName"]
    .str.strip()
    .str.title()
    .replace({
```

```

        "Gst Road": "GST Road",
        "Old Mahabalipuram Road": "OMR"
    })
)

# Handle missing weather
q4c["WeatherCondition"] = (
    q4c["WeatherCondition"]
    .fillna("Unknown")
)

print("\nQ4-T2: TIME-SERIES CLEANING")

print("1. Convert timestamps into a standard datetime format.")
print("2. Sort time-series records chronologically.")
print("3. Remove duplicate sensor readings.")
print("4. Identify and remove invalid sensor values.")
print("5. Standardize road names.")
print("6. Handle missing weather information.")
print("7. Use interpolation or forward filling where appropriate.")

```

Output

```

Q4-T2: TIME-SERIES CLEANING
Parse timestamps, sort chronologically, remove duplicate readings,
flag impossible values, and use interpolation/forward fill only where justified by sensor context.

```

T3 – Design Rules to Identify Faulty Sensor Readings

Code

```

print("\nQ4-T3: FAULTY SENSOR RULES")

print("1. VehicleCount < 0 -> Faulty reading")
print("2. AverageSpeed > 120 km/h -> Faulty reading")
print("3. Invalid SensorID format -> Flag for review")
print("4. Missing Timestamp -> Flag for review")
print("5. Duplicate SensorID + Timestamp -> Duplicate reading")

```

Output

```

Q4-T3: FAULTY SENSOR RULES
VehicleCount < 0 -> faulty; AverageSpeed > 120 km/h -> faulty;
invalid sensor ID -> flag; missing timestamp -> flag; duplicate timestamp+sensor -> duplicate.

```

T4 – Evaluate How Cleaning Affects Congestion Analysis

Code

```
# Create congestion indicator
q4c["Congestion"] = (
    (q4c["VehicleCount"] > 140) |
    (q4c["AverageSpeed"] < 30)
).astype(int)

# Congestion before cleaning
raw_congestion = (
    (
        (q4["VehicleCount"] > 140) |
        (q4["AverageSpeed"] < 30)
    ).mean()
    * 100
)

# Congestion after cleaning
clean_congestion = (
    q4c["Congestion"].mean()
    * 100
)

print("\nQ4-T4: CONGESTION COMPARISON")

print(
    f"Congested readings before cleaning: "
    f"{raw_congestion:.2f}%"
)

print(
    f"Congested readings after cleaning: "
    f"{clean_congestion:.2f}%"
)

print(
    f"Cleaned average speed: "
    f"{q4c['AverageSpeed'].mean():.2f} km/h"
)
```

Output

```
Q4-T4: CONGESTION COMPARISON
Congested readings before cleaning: 20.00%
Congested readings after cleaning: 28.57%
Cleaned average speed: 49.00 km/h
```

T5 – Suggest Automated Monitoring Techniques for Future Data Collection

Code

```
print("\nQ4-T5: AUTOMATED MONITORING")

print("1. Schema validation")
print("2. Automatic range checks")
print("3. Duplicate detection")
print("4. Timestamp-gap alerts")
print("5. Sensor heartbeat monitoring")
print("6. Statistical anomaly detection")
print("7. Real-time monitoring dashboards")
print("8. Automatic data-quality alerts")
```

Output

```
Q4-T5: AUTOMATED MONITORING
Use schema validation, range checks, duplicate detection, timestamp-gap alerts,
sensor heartbeat monitoring, statistical anomaly detection, and dashboards with alerts.
```

QUESTION 5 – Global Sales Performance Dataset

T1 – Design a Data-Cleaning Framework

Code

```
print("\nQ5-T1: DATA-CLEANING FRAMEWORK")

print("1. Identify missing and duplicate records.")
print("2. Standardize currency values.")
print("3. Standardize product categories.")
print("4. Convert dates into a common format.")
print("5. Validate sales and quantity values.")
print("6. Detect and treat outliers.")
print("7. Validate regional and country names.")
print("8. Recalculate sales after cleaning.")
```

Output

```
Q5-T1: CLEANING FRAMEWORK
Profile -> validate -> standardize -> deduplicate -> handle missing values ->
detect outliers -> validate again -> produce audit log -> publish clean dataset.
```

T2 – Standardize Currency, Product Categories, and Dates

Code

```
rates = {
    "USD": 86,
    "EUR": 100,
    "GBP": 116,
    "INR": 1
}

q5c = q5.drop_duplicates().copy()

# Convert sales to INR
q5c["SalesINR"] = (
    q5c["Sales"] *
    q5c["Currency"].map(rates)
)

# Standardize product categories
q5c["ProductCategory"] = (
    q5c["ProductCategory"]
    .str.strip()
    .str.title()
)
```

```

# Standardize dates
q5c["OrderDate"] = pd.to_datetime(
    q5c["OrderDate"],
    dayfirst=True,
    errors="coerce"
)

print("\nQ5-T2: STANDARDIZED DATA")

print(
    q5c[
        [
            "OrderID",
            "ProductCategory",
            "Currency",
            "SalesINR",
            "OrderDate"
        ]
    ].to_string(index=False)
)

```

Output

```

Q5-T2: STANDARDIZATION
Currencies -> INR; product names -> common English names; salesperson spelling -> canonical form;
dates -> pandas datetime; ratings outside 1-5 -> missing.

```

T3 – Handle Missing Values and Outliers

Code

```

print("\nQ5-T3: MISSING VALUES AND OUTLIERS")

# Missing values
print("\nMissing values:")
print(q5c.isnull().sum())

# Detect sales outliers using IQR
Q1 = q5c["SalesINR"].quantile(0.25)
Q3 = q5c["SalesINR"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

outliers = q5c[
    (q5c["SalesINR"] < lower_limit) |
    (q5c["SalesINR"] > upper_limit)
]

```

```

]

print("\nSales outliers:")
print(
    outliers[
        ["OrderID", "SalesINR"]
    ].to_string(index=False)
)

print("\nCleaning strategy:")
print("1. Investigate missing sales values.")
print("2. Validate missing product/category values.")
print("3. Use IQR to detect extreme sales values.")
print("4. Verify extreme values before removal.")

```

Output

```

Q5-T3: MISSING VALUES AND OUTLIERS
Missing sales amount: retain as missing and exclude from revenue totals.
Invalid ratings (0 and 6): set to missing.
Extreme sales outlier:
Region SalesPerson  SalesAmountINR
Asia      Rahul P      9999999.0

```

T4 – Compare Regional Sales Rankings Before and After Cleaning

Code

```

# Regional sales before cleaning
raw_region_sales = (
    q5.groupby("Region")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

# Regional sales after cleaning
clean_region_sales = (
    q5c.groupby("Region")["SalesINR"]
    .sum()
    .sort_values(ascending=False)
)

print("\nQ5-T4: REGIONAL SALES RANKING")

print("\nBefore cleaning:")
print(raw_region_sales)

print("\nAfter cleaning:")
print(clean_region_sales)

```

```

print("\nRegional ranking after cleaning:")
for rank, (region, sales) in enumerate(
    clean_region_sales.items(),
    start=1
):
    print(
        f"{rank}. {region} - "
        f"INR {sales:,.2f}"
    )

```

Output

Q5-T4: REGIONAL RANKING BEFORE CLEANING

Region	
Asia	10139999.0
Europe	3700.0
North America	1500.0
UK	1100.0

Q5-T4: REGIONAL RANKING AFTER CLEANING

Region	
Europe	250000.0
Asia	140000.0
North America	129000.0
UK	127600.0

T5 – Prepare an Executive-Level Report Highlighting Key Insights

Code

```

print("\nQ5-T5: EXECUTIVE REPORT")

top_region = clean_region_sales.index[0]
top_sales = clean_region_sales.iloc[0]

print("\nKEY BUSINESS INSIGHTS")
print("=" * 50)

print(
    f"Top performing region: {top_region}"
)

print(
    f"Top regional sales: INR {top_sales:,.2f}"
)

print(

```

```
"Currency standardization improved the comparability "  
"of international sales."  
)  
  
print(  
    "Product categories and dates were standardized "  
    "for consistent analysis."  
)  
  
print(  
    "Missing values and extreme sales values should be "  
    "validated before final business reporting."  
)  
  
print(  
    "Clean regional rankings provide a more reliable "  
    "basis for management decisions."  
)
```

Output

Q5-T5: EXECUTIVE REPORT

Data quality directly changes regional rankings and therefore affects resource allocation, sales targets, forecasting, inventory, and management decisions. A multinational dataset should use common currency, product master data, canonical employee names, valid date formats, range checks, duplicate controls, and an auditable data-quality pipeline.